# SISTEMAS OPERATIVOS I

## Práctica 4: Utilización de hilos

## Objetivo

Introducción al manejo de hilos y a su funcionamiento.

## Información útil para la práctica

- *Los hilos de un proceso comparten el espacio de memoria*: las variables **globales** se comparten entre los hilos, mientras que las locales al hilo principal o a los otros hilos que se crean no se deben compartir, son **locales** a cada hilo. De todas formas, cualquier hilo puede acceder a cualquier variable del proceso a través de su dirección (puntero).
- Lee el manual de uso de *pthread_ create* para crear los hilos y de *pthread_ exit* para finalizarlos.
- Lee el manual de uso de la llamada al sistema *pthread_ join* para esperar a la finalización de un hilo.
- Al compilar acuérdate de incluir la opción de enlazado: ***-lpthread***.

## Enunciado

1. Haz un programa para comprobar que tanto las variables globales como las locales a los hilos son accesibles por todos ellos. Comprueba que si un hilo cambia una de esas variables, los demás pueden comprobar dicho cambio.
2. Haz un programa que cree tres hilos y que el último de ellos cree un proceso hijo con *fork*. Comprueba con *ps* cuántos hilos tiene ese proceso hijo, es decir, si se crea también con tres hilos o no. Comprueba que ocurre con este proceso hijo cuando termina el hilo desde el que se ha creado (sin cambiar la imagen del proceso). En el código debes garantizar que cuando el tercer hilo ejecuta el *fork*, los demás no han acabado.
3. Haz un programa con tres hilos (además del hilo principal) de modo que uno de ellos ejecute *exit* (no *pthread_ exit*) antes que los otros dos hilos terminen y comprueba que efecto tiene este *exit* sobre los demás hilos. Debes garantizar que los hilos no finalizan previamente a la ejecución de dicho *exit*.
4. Haz un programa para comprobar el efecto de la función *pthread_ yield*. Una forma sencilla de hacer esto es crear un programa con varios hilos, de modo que algunos sean mucho más generosos que los damás liberando la CPU muy frecuenemente. Se trataría de comprobar que cuando se hace la llamada a esa función, los hilos generosos acaban más tarde que los que no lo son. Si se te ocurren situaciones en las que puede ser beneficioso usar *pthread_ yield* en un proceso multihilo, pruébalas y coméntalas al profesor.
5. Haz un programa que calcule el valor del número $\pi$ mediante la fórmula de Adamchik y Wagon:

$$\pi = \sum_{k=0}^{\infty} \frac{(-1)^k}{4^k} \left( \frac{2}{4k+1} + \frac{2}{4k+2} + \frac{1}{4k+3} \right)$$

   para $k$ desde 0 hasta 10000000 usando varios hilos y de acuerdo con las siguientes especificaciones:

(a) El número de hilos $H$ debe estar definido como una constante en el código. Busca información sobre el número de cores de tu equipo para conocer las limitaciones hardware de tusistema. El número de hilos puede variar entre 1 y el máximo número de cores.

(b) El $j$-ésimo hilo $H_j$ realiza el cálculo de un grupo de 100 iteraciones consecutivas del sumatorio, haciendo un reparto cíclico de los grupos entre todos los hilos. Por ejemplo, si $H = 4$, el hilo $H_2$ hace el cálculo de las iteraciones $[100..199, 500..599, 900..999, ...]$. Comprueba que el número total de iteraciones es correcto, por ejemplo, cambiando el sumatorio de la fórmula por el sumatorio de las iteraciones. Después de comprobarlo, comenta el código para dejarlo en el entregable sin que afecteal resto de los apartados.

(c) Todos loos hilos trabajan de manera concurrente, haciendo las operaciones en doble precisión.

(d) Cada uno de los hilos hace su cálculo sobre una variable sumatorio diferente para evitar carreras críticas.

(e) El hilo principal debe esperar a que acaben todos los hilos para calcular el sumatorio total sumando las contribuciones parciales de cada hilo para obtener el valor final de $\pi$.

(f) Trás todo ello, el hilo principal hace el sumatorio completo en secuencial, y muestra los resultados de $\pi$ de las versiones paralela y secuencial así como la diferencia entre ambas.

(g) Consulta el monitor del sistema para comprobar que todos los cores están trabajando al ejecutar el código, y comprueba cuanto tiempo se ahorra frente a la versión secuencial.

(h) Este código constituye el único **ENTREGABLE** de la práctica.